

UNICESUMAR

Aula 007 - Expressões, Atribuição e Controle

Alunos:

Eduardo Viana Marques – 24023663-2

Leonardo Borges Fratoni – 24047610-2

Maringá

02/10/2026

Página 4 — Quiz de abertura: o que cada linha imprime?

1- Pyth

```
JS indexjs X + Console I/O AI Agent 145 ms ✓
1 console.log(0 == "", "" == "0", 0 == "0");
2
```

true false true

on

```
main.py X + Console I/O AI Agent 124 ms ✓
1 print(-2 ** 2, 2 ** 3 ** 2)
2
```

-4 512

2- Java

```
J Main.java X + Console I/O AI Agent 985 ms ✓
1 import java.util.*;
2
3 public class Main {
4     public static void main(String[] args) {
5         System.out.println(1 + 2 + "3" + 4 + 5);
6     }
7 }
8 }
```

3345

3- JavaScript

```
JS indexjs X + Console I/O AI Agent 134 ms ✓
1 console.log("5" - 2, "5" + 2);
2
```

3 52

4 – C

```
C Main.c X + Console I/O AI Agent 174 ms ✓
1 #include <stdio.h>
2 int main()
3 {
4     printf("%d\n", 3 > 2 > 1);
5 }
6 }
```

0

5– Python

```
main.py X + Console I/O AI Agent 133 ms ✓
1 x = 0
2 print(x or "padrão", 3 > 2 > 1)
3
```

padrão True

6- JavaScript

```
JS indexjs X + Console I/O AI Agent 145 ms ✓
1 console.log(0 == "", "" == "0", 0 == "0");
2
```

true false true

Página 5 — Expressões aritméticas: questões de projeto

As expressões aritméticas envolvem regras de precedência, associatividade e ordem de avaliação dos operandos. Essas regras determinam como os cálculos acontecem em cada linguagem. O uso de parênteses pode tornar a intenção mais clara.

Expressões aritméticas: questões de projeto

- Quais são as regras de **precedência**?
- Quais são as regras de **associatividade**?
- Em que **ordem os operandos** são avaliados?
- Há restrições a **efeitos colaterais**?
- O usuário pode **sobrecarregar** operadores?
- Que **mistura de tipos** é permitida?

Lisp dispensa a precedência (p. 306)

(+ a (* b c)): notação prefixa, e os parênteses dizem a ordem.

$a + b * c$ com $a=3, b=4, c=5$

da esquerda para a direita

$$(3 + 4) * 5 = 35$$

da direita para a esquerda

$$3 + (4 * 5) = 23$$

com precedência (* antes de +)

$$3 + 4 * 5 = 23$$

APL: sempre da direita para a esquerda (p. 304)

$$3 \times 4 + 5 = 27$$

Página 6 — Precedência e associatividade na prática

A precedência define quais operadores têm prioridade, enquanto a associatividade determina o agrupamento das operações com a mesma prioridade. Essas regras podem variar entre linguagens, fazendo uma expressão produzir resultados diferentes.

Precedência e associatividade na prática

Expressão	Python	JavaScript	Java
-2 ** 2	-4	erro de sintaxe	(não há **)
2 ** 3 ** 2	512	512	—
1 + 2 + "3"	TypeError	"33"	"33"
-7 / 2 (inteiros)	-4 (com //)	-3.5	-3
-7 % 2	1	-1	-1

Associatividade (p. 303)

Esquerda: * / % + -
Direita: ** (Python, JS, Ruby, Fortran),
++, --, menos unário
Visual Basic: ^ à esquerda!

Excel

=-2^2 dá 4: lá o menos unário vence o
^.

Na dúvida, **use parênteses** (p. 305): eles deixam a intenção explícita para o compilador e para o próximo leitor.

Página 7 — Ordem dos operandos e efeitos colaterais

A ordem de avaliação pode alterar o resultado quando uma função modifica variáveis utilizadas na expressão. Java estabelece a avaliação da esquerda para a direita. Evitar efeitos colaterais dentro de expressões facilita a compreensão do código.

Ordem dos operandos e efeitos colaterais

Exemplo do livro (p. 307)

```
int a = 5;

int fun1(void) {
    a = 17;      /* efeito colateral */
    return 3;
}

int main(void) {
    a = a + fun1(); /* 8 ou 20? */
    printf("a = %d\n", a);
}
```

8

Java

a ordem é garantida: da esquerda para a direita (p. 308)

20

C (gcc 14)

a ordem não é definida pelo padrão

20

Go

a especificação não define a leitura de a em relação à chamada

Transparência referencial (p. 308): sem efeitos colaterais, a ordem deixa de importar, e $f(x) + f(x)$ pode virar $2 * f(x)$.

Página 8 — Operadores sobrecarregados

A sobrecarga permite que um operador tenha diferentes significados conforme os tipos envolvidos. O símbolo +, por exemplo, pode somar números ou concatenar textos. Algumas linguagens também permitem definir operadores para classes criadas pelo programador.

Operadores sobrecarregados

- O mesmo símbolo com **vários significados**: + soma inteiros, reais e concatena (Java)
- Risco (p. 309): & em C++ é E bit a bit e "endereço de"
- **Definida pelo usuário** (p. 404): C++, C#, Python, Rust, Kotlin. Java **não** permite

Java

```
System.out.println('a' + 1); // 98
System.out.println("" + 'a' + 1); // a1
x.add(y).multiply(tres); // BigDecimal: sem +
```

Python · sobrecarga com métodos especiais

```
class Vetor:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __add__(self, outro): # v1 + v2
        return Vetor(self.x + outro.x,
                      self.y + outro.y)

    def __mul__(self, k): # v * 3
        return Vetor(self.x * k, self.y * k)

v = Vetor(1, 2) + Vetor(3, 4) # Vetor(4, 6)
print(v * 3) # Vetor(12, 18)
```

Página 9 — Conversões de tipos: alargamento, estreitamento e coerção

As conversões transformam valores entre tipos diferentes e podem acontecer automaticamente ou de maneira explícita. O estreitamento pode descartar informações, e até algumas conversões de alargamento podem perder precisão. Por isso, é necessário verificar os tipos envolvidos nos cálculos.

Conversões de tipos: alargamento, estreitamento e coerção

Java · coerção e cast

```
int grande = 123456789;
float f = grande;           // alargamento implícito
System.out.println((int) f); // 123456792 !

double d = 3.99;
System.out.println((int) d); // 3 (cast)
System.out.println((byte) 200); // -56

byte b = 10, c = 20;
// byte s = b + c; // erro: b + c é int (p. 312)

System.out.println(1 / 2); // 0
System.out.println(1 / 2.0); // 0.5
```

Linguagem	int → real (implícito)	int32 + int64
C	sim	coerção
Java	sim (alargamento)	coerção para long
Python	sim	(um só tipo int)
Go	não: float64(i)	erro de compilação
Rust	não: i as f64	erro de compilação
ML, F# (p. 312)	não	erro de compilação

Coerção esconde erros (p. 312): em `d = b * a`; com `a` digitado por engano no lugar de `c`, o Java converte e compila.

Página 10 — Coerção implícita: JavaScript e PHP

JavaScript e PHP realizam conversões automáticas que podem gerar resultados inesperados. Em JavaScript, `"5" - 2` resulta em 3, enquanto `"5" + 2` produz `"52"`. Nas comparações, os operadores `===` e `!==` evitam a coerção entre tipos.

Coerção implícita: JavaScript e PHP

JavaScript

```
"5" - 2           // 3
"5" + 2           // "52"
"5" * "2"         // 10
true + 1          // 2
[] + {}           // "[object Object]"
null + 1          // 1
undefined + 1     // NaN
"b" + "a" + "a" + "a" // "baNaNa"

0 == ""           // true
"" == "0"         // false
0 == "0"          // true (não é transitivo!)
0 === "0"         // false (=== não converte)
```

PHP · "type juggling"

```
var_dump("0e123" == "0e456"); // true !
var_dump("0e123" === "0e456"); // false
var_dump("1" == "01"); // true

md5("240610708") // "0e462097431906...."
md5("QNKCDZO") // "0e830400451993...."
// == entre os dois hashes: true
```

Regra prática

Em JavaScript e PHP, compare com `===` e `!==`, que **não** fazem coerção.

Página 11 — Expressões relacionais e booleanas

As expressões relacionais comparam valores, enquanto as booleanas combinam condições. Cada linguagem possui regras próprias para interpretar valores como verdadeiros ou falsos. Comparações encadeadas também exigem atenção, pois possuem significados diferentes em Python e C.

Expressões relacionais e booleanas

C · booleano é inteiro (p. 315)

```
int a = 3, b = 2, c = 1;
printf("%d\n", a > b > c);    // 0 !
// (3 > 2) vale 1, e 1 > 1 é falso

printf("%d\n", (a > b) && (b > c));    // 1

if (-1) printf("verdadeiro\n");    // != 0
```

Python · comparações encadeadas

```
a, b, c = 3, 2, 1
print(a > b > c)    # True: (a > b) and (b > c)

# contam como FALSO em um if:
# 0    0.0    ""    []    {}    None
# contam como VERDADEIRO:
# "0"    [0]    " "
```

Linguagem	O que o if aceita
C, Python, JavaScript	qualquer valor (0, vazio e nulo contam como falso)
Java, C#, Go, Rust	somente booleano : if (n) com n inteiro nem compila

Página 12 — Avaliação em curto-circuito

O curto-circuito interrompe a avaliação quando o resultado lógico já está determinado. Em uma expressão com &&, uma primeira condição falsa impede a avaliação da segunda. Esse recurso permite verificar um objeto antes de acessá-lo e ajuda a evitar erros.

Avaliação em curto-circuito

Java · && faz curto-circuito, & não

```
// busca do livro (p. 316)
while (i < lista.length && lista[i] != chave)
    i++;           // && evita lista[lista.length]

String s = null;
s != null && s.length() > 0    // false
s != null & s.length() > 0    // NullPointerException
```

JavaScript · ?. e ??

```
const u = { nome: "Ana", endereco: null };
u.endereco?.cidade    // undefined, sem erro

const qtd = 0;
qtd || 10              // 10 (o 0 se perdeu!)
qtd ?? 10              // 0  (só troca null/undefined)
```

Cuidado com efeitos colaterais (p. 316)
Em (a > b) || (b++ / 3), o b++ só acontece quando a <= b. Em Python, 0 or "padrão" devolve "padrão": and e or devolvem um dos **operandos**.



Página 13 — Sentenças de atribuição

A atribuição associa um valor a uma variável. Em algumas linguagens, ela também pode aparecer dentro de expressões, exigindo cuidado para não confundir atribuição com comparação. A aula apresentou ainda a atribuição múltipla e a troca de valores em Python.

Sentenças de atribuição

C · a atribuição é uma expressão (p. 319-320)

```
sum = count = 0;           // múltiplos alvos
a = b + (c = d / b) - 1;    // exemplo do livro

while ((ch = getchar()) != EOF) { ... }

if (x = y) { ... }    // compila! atribui e testa y
```

Linguagem	Atribuição é expressão?
C, C++, JavaScript	sim: if (x = y) compila
Java, C#	sim, mas o if exige booleano
Python	não, exceto o operador morsa :=
Go	não: nem x++ é expressão
Rust	vale (): if x = 5 não compila

Python · atribuição múltipla (p. 320)

```
a, b = b, a                # troca sem temporária
if (n := len(dados)) > 5: ... # morsa (Python 3.8+)
```

Linguagens funcionais (p. 321)
val (ML) e let (F#, Rust) dão **nome** a um valor, que não muda.



Página 14 — Atribuição de modo misto

A atribuição de modo misto envolve valores e variáveis de tipos diferentes. O cálculo ocorre antes da conversão para o tipo de destino. Assim, uma divisão entre inteiros perde a parte decimal mesmo quando seu resultado é atribuído a uma variável double.

Atribuição de modo misto

Java - só alargamento (p. 322)

```
double d = 3;           // ok: alargamento
// int i = 3.7;         // erro: estreitamento
int i = (int) 3.7;       // 3 (cast explícito)
long l = i;              // ok

i += 3.7;                // COMPILA: cast escondido
System.out.println(i);   // 6
```

C - coerção em qualquer direção

```
int i = 3.7;             // 3, em silêncio
char c = 300;            // 44 (só um aviso)
unsigned int u = -1;      // 4294967295
```

Armadilha clássica

`double taxa = acertos / total * 100;` com inteiros dá **0.0**: a divisão inteira acontece **antes** do alargamento para double.

Página 15 — Estruturas de controle: quantas são necessárias?

Sequência, seleção e iteração são as estruturas fundamentais para organizar algoritmos. Elas permitem executar instruções em ordem, escolher caminhos e repetir operações. Estruturas como `for` e `switch` oferecem formas adicionais de expressar essas ações com clareza.

Estruturas de controle: quantas são necessárias?

Sequência

uma sentença após a outra

```
a = 1;
b = a + 2;
```

Seleção

escolher um caminho

```
if (x > 0) ...
else ...
```

Iteração

repetir enquanto uma condição vale

```
while (x > 0)
    x--;
```

Böhm e Jacopini (1966): sequência, seleção e laço lógico bastam para qualquer algoritmo. O goto não é necessário.

Por que então tantas outras (for, switch, do, foreach)? **Facilidade de escrita.** O limite é a **legibilidade**: quanto maior a linguagem, menos do código alheio você reconhece (p. 329).

Página 16 — Seleção de dois caminhos: o "else pendente"

As estruturas if e else escolhem caminhos conforme uma condição. Quando existem condições aninhadas, o else pertence ao if mais próximo que ainda não possui alternativa. O uso de chaves evita interpretações equivocadas sobre essa associação.

Seleção de dois caminhos: o "else pendente"

C - a endentação engana (p. 331)

```
int sum = 0, count = 5, result = -1;
if (sum == 0)
    if (count == 0)
        result = 0;
else
    result = 1;           // de qual if é este else?
printf("%d\n", result);  // 1
```

Regra de Java, C, C++ e C#

O else pertence ao if **mais próximo** que ainda não tem else. A endentação é ignorada (exceto em Python e F#).

Chaves obrigatórias eliminam o problema: Perl, Go e Rust (p. 332).

Seletores que devolvem valor

```
let y = if x > 0 { x } else { 2 * x };
y = x if x > 0 else 2 * x
```

// Rust: if é uma expressão (como em F#, p. 334)
Python: expressão condicional (p. 306)

Página 17 — Seleção múltipla: do switch de C ao match

A seleção múltipla permite escolher entre várias alternativas. No switch tradicional, a ausência de break pode provocar a execução dos casos seguintes. Outras estruturas, como o match de Rust, impedem essa continuidade automática e exigem cobertura dos valores possíveis.

Seleção múltipla: do switch de C ao match

C · sem break, o controle "cai" (p. 335)

```
switch (index) {
  case 1:
  case 3: odd += 1;
  case 2:
  case 4: even += 1;
  default: printf("Error\n");
}
// index = 1: odd, even E erro!
```

Java 14+ · switch como expressão

```
String tipo = switch (dia) {
  case SAB, DOM -> "fim de semana";
  case SEG, TER, QUA,
    QUI, SEX -> "dia útil";
}; // sem break, sem cair;
// exaustivo para enum
```

Rust · match exaustivo

```
match n {
  i32::MIN..=-1 => "negativo",
  0 => "zero",
  1..=9 => "um dígito",
  _ => "vários dígitos",
}
// sem o _ : não compila
```

Go: cada caso termina sozinho, e cair exige `fallthrough`. C#: cair implicitamente é proibido (p. 336). Python 3.10: `match` com padrões e guardas.

Página 18 — Laços controlados por contador

Os laços controlados por contador repetem operações conforme uma sequência de valores. Em C, modificar o contador no corpo pode alterar as próximas repetições. Em Python, o `for` obtém cada novo valor de um iterável, independentemente de alterações feitas na variável durante a repetição.

Laços controlados por contador

C - o for é um laço lógico disfarçado

```
for (int i = 0; i < 10; i++) {  
    printf("%d ", i);  
    i += 2;           // altera a contagem  
}  
// 0 3 6 9  
  
for (; i <= 100; ) // expressões opcionais  
    soma += i++;
```

Python - o for percorre um iterável

```
for i in range(0, 10):  
    print(i, end=" ")  
    i += 2           # NÃO muda nada  
# 0 1 2 3 4 5 6 7 8 9  
  
for i in range(10, 0, -3): # 10 7 4 1  
    ...
```

Questões de projeto (p. 342)

Qual é o tipo e o escopo da variável? Posso alterá-la no corpo? Os parâmetros são avaliados uma vez ou a cada volta?

Página 19 — Laços lógicos, saídas e iteradores

Laços lógicos repetem instruções conforme uma condição. O while verifica a condição antes da execução, enquanto o do-while executa o corpo pelo menos uma vez. A aula também apresentou saídas com break, iteradores e a importância de garantir progresso para evitar laços infinitos.

Laços lógicos, saídas e iteradores

Java - break rotulado (p. 348)

```
externo:  
for (int i = 0; i < m.length; i++)  
    for (int j = 0; j < m[i].length; j++)  
        if (m[i][j] < 0) {  
            System.out.println(i + ", " + j);  
            break externo; // sai dos DOIS laços  
        }
```

JavaScript - for...in x for...of

```
const notas = [10, 20, 30];  
let total = 0;  
for (const i in notas) total += i;  
console.log(total); // "0012": chaves!  
  
total = 0;  
for (const v of notas) total += v;  
console.log(total); // 60: valores
```

Zune (2008) - o laço lógico sem progresso

```
while (dias > 365) {  
    if (bissexto(ano)) {  
        if (dias > 366) { dias -= 366; ano++; }  
        // dias == 366: nada muda, laço infinito!  
    } else { dias -= 365; ano++; }  
}
```

Pós-teste (do-while) executa **ao menos uma vez**, e isso é fácil de esquecer (p. 347).

Página 20 — Desvio incondicional: o goto

O goto transfere a execução diretamente para um ponto identificado do programa. Seu uso pode dificultar o acompanhamento do fluxo e favorecer erros. Comandos como break, continue e return oferecem formas mais delimitadas de alterar a execução.

Desvio incondicional: o goto

"goto fail" (Apple, 2014) - versão simplificada

```
int valida_certificado(void) {
    int err;
    if ((err = verifica_hash()) != 0)
        goto fail;
    goto fail;           // duplicada: desvia SEMPRE
    if ((err = verifica_assinatura()) != 0)
        goto fail;       // nunca é alcançada
fail:
    return err;           // 0 = "certificado válido"
}
```

Dijkstra (1968)

"Go To Statement Considered Harmful"

Quem tem goto hoje

C, C++, C#, Go. Java **não** tem (p. 353)

Gotos restritos

break, continue e return melhoram a legibilidade (p. 353)

Com chaves obrigatórias (Go, Rust) ou com gcc -Wall (aviso de endentação enganosa), o bug teria sido pego.

Página 21 — Comandos protegidos de Dijkstra

Os comandos protegidos associam ações a condições chamadas guardas. Quando várias guardas estão habilitadas, uma delas pode ser escolhida de maneira não determinística. A aula relacionou esse conceito ao select de Go, utilizado para selecionar operações com canais.

Comandos protegidos de Dijkstra

```
if x >= y -> max := x
[] y >= x -> max := y
fi
```

Exemplo do livro (p. 354): se $x = y$, qualquer ramo serve.

- Todas as **guardas** são avaliadas
- Várias verdadeiras: escolha **não determinística**
- Nenhuma verdadeira: erro (no if) ou fim do laço (no do ... od)
- Base do **CSP** de Hoare (1978) e do select de Go

Go · select, herdeiro dos comandos protegidos

```
a <- "a"    // os dois canais
b <- "b"    // estão prontos

select {    // Go sorteia uma guarda
case x := <-a:
    contagem[x]++
case x := <-b:
    contagem[x]++
}

// 1000 repetições:
// map[a:505 b:495] (muda a cada execução)
```

Página 22 — Exercício em duplas: encontre o bug (1/3)

Os exercícios abordaram o uso incorreto de for...in para somar valores em JavaScript e a substituição indevida de um desconto zero em Python. As correções envolvem percorrer os valores com for...of e verificar explicitamente se o desconto é None.

Exercício em duplas: encontre o bug (1/3)

Para cada trecho: **(a)** qual é a saída? **(b)** qual regra causa o problema? **(c)** como corrigir?

1 · JavaScript

```
const precos = [10, 20, 30];
let total = 0;
for (const p in precos)
    total += p;
console.log("Total: " + total);
```

2 · Python

```
def desconto(preco, d=None):
    d = d or 10    # padrão: 10%
    return preco * (100 - d) / 100

print(desconto(200, 0))
```

Página 23 — Exercício: encontre o bug (2/3)

Os exemplos mostraram erros causados pela ausência de break em um switch e pelo ponto e vírgula após um if. Essas situações demonstram como pequenos detalhes da sintaxe podem alterar o fluxo de execução.

Exercício: encontre o bug (2/3)

Para cada trecho: **(a)** qual é a saída? **(b)** qual regra causa o problema? **(c)** como corrigir?

3 · Java

```
int dia = 2; String nome = "";
switch (dia) {
    case 1: nome = "domingo";
    case 2: nome = "segunda";
    case 3: nome = "terça";
    default: nome = "inválido";
}
System.out.println(nome);
```

4 · C

```
int saldo = 100, saque = 50;
if (saque > saldo);
    printf("saldo insuficiente\n");
printf("fim\n");
```

Página 24 — Exercício: encontre o bug (3/3)

Os exercícios trataram da declaração de uma variável local que oculta outra externa em Go e da divisão inteira no cálculo de porcentagens em Java. As soluções exigem atualizar a variável existente e converter um operando antes da divisão.

Exercício: encontre o bug (3/3)

Para cada trecho: **(a)** qual é a saída? **(b)** qual regra causa o problema? **(c)** como corrigir?

5 · Go

```
x := 10
if x > 5 {
    x := x * 2
    fmt.Println("dentro:", x)
}
fmt.Println("fora:", x)
```

6 · Java

```
int acertos = 7, total = 10;
double taxa = acertos / total * 100;
System.out.println(taxa + "%");
```

Página 25 — Síntese: armadilhas e como as linguagens ajudam

A aula reuniu problemas relacionados a expressões, conversões e estruturas de controle. Recursos como comparações estritas, conversões explícitas e verificação das alternativas ajudam a reduzir erros. Conhecer essas regras permite escrever códigos mais claros e previsíveis.

Síntese: armadilhas e como as linguagens ajudam

Tema	Armadilha	Como as linguagens ajudam
Precedência	<code>-2 ** 2, 1 + 2 + "3"</code>	parênteses; JS obriga <code>(-2) ** 2</code>
Operandos	<code>a + f()</code> com efeito colateral (C, Go)	Java fixa a ordem; funções puras
Conversões	coerção silenciosa (C, JS, PHP)	Go e Rust exigem conversão explícita; <code>===</code>
Booleanos	<code>0, "", []</code> são falsos; <code>a > b > c</code> em C	<code>if</code> só com booleano (Java, Go, Rust); <code>??</code>
Atribuição	<code>if (x = y)</code>	atribuição não é expressão (Python, Go, Rust)
Seleção	<code>switch</code> sem <code>break</code> ; <code>else</code> pendente	<code>-></code> , <code>match</code> exaustivo, chaves obrigatórias
Laços	<code>for...in</code> no JS; laço sem progresso	<code>for...of</code> , iteradores, <code>break</code> rotulado
Desvio	<code>goto</code> sem restrição	<code>break</code> , <code>continue</code> , <code>return</code> : gotos restritos

Página 26 — Atividades complementares (autopesquisa)

As atividades propõem aprofundar o estudo de expressões, seleção múltipla e repetição em diferentes linguagens. A comparação entre `switch` e `match` permite analisar a continuidade entre casos, a cobertura das alternativas e a legibilidade das soluções.

Atividades complementares (autopesquisa)

- **Expressões em Ruby e em Lisp** (§7.2.1.4-7.2.1.5, p. 305-306):
`expressoes.rb`, `expressoes.lisp`
- **Expressões condicionais** (§7.2.1.6, p. 306) e **erros em expressões** (§7.4.3, p. 313)
- **Implementação da seleção múltipla**: tabelas de desvio (§8.2.2.3, p. 337-339)
- **Laços em linguagens funcionais**: contagem por recursão (§8.3.1.4, p. 345)
- **Iteradores e blocos de Ruby** (§8.3.4, p. 349-352)
- Leitura: Dijkstra, "Go To Statement Considered Harmful" (1968)

Entrega sugerida

Um mesmo exemplo com `switch` em C, `switch` -> em Java, `switch` em Go e `match` em Rust: compare "cair" entre casos, exaustividade e legibilidade.

Próxima aula

Subprogramas (Caps. 9 e 10)